

GreenStep Employee Sustainability Challenge

Source Code Documentation

Oli Gurmessa

Hadi Shaar

Naisha Srivastav

Nathanael Agbemadon

CISC480 — Senior Capstone

Instructor: Dr. Jimenez • Spring 2026

Submitted: May 22, 2026

Contents

1 Overview and Tooling	2
2 Project / Folder Structure	2
3 Naming Conventions	3
4 Imports and Module Conventions	3
5 Server vs. Client Components	3
6 API Route Pattern	4
7 Commenting	4
8 Formatting (Prettier)	5
9 Version Control	5

1 Overview and Tooling

This document describes the code-construction guidelines for the GreenStep codebase: project structure, naming, imports, the server/client component model, the API route pattern, commenting, formatting, and version-control practice. The conventions follow the Google TypeScript Style Guide [1] adapted to a Next.js 15 + TypeScript project. Every example below cites a real file in the repository so the rule is traceable to the code.

Two tools enforce quality, and both must pass before a pull request is merged:

- **Prettier** [2] for formatting, configured by `.prettierrc` at the repository root.
- **ESLint** [3] for linting, configured by `eslint.config.mjs` (the Next.js flat-config preset `next/core-web-vitals` + `next/typescript`), run via `npm run lint`.

The continuous-integration gate is `npm run build` and `npm run lint`. Automated tests are not yet configured for the project; test plans and cases are maintained in the QA Activities document and listed as future work in the Project Report.

2 Project / Folder Structure

The repository follows the Next.js App Router layout:

Table 1: Top-level source directories and their responsibilities.

Path	Responsibility
<code>app/</code>	Routed pages and layouts (Presentation). Route groups (<code>auth/</code> and <code>dashboard/</code>) organize related screens.
<code>app/api/</code>	Server-side API route handlers (<code>route.ts</code> files).
<code>app/actions/</code>	Server actions (e.g. <code>challenge.ts</code>).
<code>lib/</code>	Framework-agnostic helpers: <code>db.ts</code> (Prisma singleton), <code>authz.ts</code> , <code>constants.ts</code> , <code>global-challenge.ts</code> , <code>team-requests.ts</code> , <code>actions-data.ts</code> .
<code>components/</code>	Shared, reusable UI components imported across pages.
<code>prisma/</code>	<code>schema.prisma</code> — the database schema.
<code>scripts/</code>	Developer seed scripts (<code>seed-*.js</code>).
<code>middleware.ts</code>	Clerk route protection (root).

3 Naming Conventions

Table 2: Naming conventions with a real example from the codebase.

Case	Used for	Example (file)
PascalCase	Components, types, interfaces	Home in <code>app/page.tsx</code>
camelCase	Functions, variables, params	<code>createTeamRequest</code> , <code>reviewTeamRequest</code> in <code>lib/team-requests.ts</code>
UPPER_CASE	Module constants	<code>GLOBAL_CHALLENGE_ID</code> in <code>lib/constants.ts</code>
kebab-case	Route folders	<code>app/(dashboard)/log-action/</code> , <code>app/api/team/request/</code>

A component file is named after its main export (e.g. `DashboardClient.tsx` exports `DashboardClient`). Vague names such as `utils.ts` are avoided in favor of scoped names (`team-requests.ts`, `global-challenge.ts`).

4 Imports and Module Conventions

The project uses the `@/` path alias, configured in `tsconfig.json` as `"@/*": ["./*"]`, so modules are imported by absolute path rather than long relative chains.

Database access goes through a single Prisma client singleton in `lib/db.ts`, imported everywhere as `db`:

```
// lib/db.ts
export const db =
  globalForPrisma.prisma ??
  new PrismaClient({ log: [...] });
if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = db;

// usage in any route or lib module:
import { db } from "@/lib/db";
```

The singleton prevents exhausting database connections during development hot-reload.

5 Server vs. Client Components

Components are **server components by default**. Data fetching, auth, and Prisma access happen on the server; for example, `app/(dashboard)/dashboard/page.tsx` authenticates and syncs the user before rendering.

A component that needs browser APIs, state, or event handlers opts in with the `"use client"` directive on its first line — for example, `app/(dashboard)/dashboard/DashboardClient.tsx`:

```
"use client";
import { useState, useEffect } from "react";
// ... interactive dashboard
```

Server-only mutations are written as **server actions** with the `"use server"` directive, e.g. `app/actions/challenge` (`createChallenge`, `joinChallenge`).

6 API Route Pattern

Every API route handler follows the same shape: a `try/catch` wrapping the logic, an authentication guard, input validation, and `NextResponse.json(...)` with an explicit HTTP status code. The pattern below is taken from `app/api/actions/route.ts`:

```
export async function POST(req: Request) {
  try {
    const { userId } = await auth();
    if (!userId)
      return NextResponse.json({ error: "Unauthorized" }, { status: 401 });

    const { category, actionType } = await req.json();
    if (!category || !actionType)
      return NextResponse.json(
        { error: "category and actionType are required" },
        { status: 400 });

    const user = await db.user.findUnique({ where: { clerkId: userId } });
    if (!user)
      return NextResponse.json({ error: "User not found" }, { status: 404 });

    // ... perform work ...
    return NextResponse.json({ ...action, streak: newStreak });
  } catch (error) {
    console.error(error);
    return NextResponse.json(
      { error: "Failed to create action" }, { status: 500 });
  }
}
```

Status codes are used consistently across handlers: 401 unauthorized, 400 bad input, 404 not found, 409 conflict (e.g. the duplicate-action guard), and 500 for unexpected errors. The same shape appears in `app/api/leaderboard/route.ts` and `app/api/me/route.ts`.

7 Commenting

Exported functions and components are documented with JSDoc. For functions with parameters, a multi-line block documents each `@param`; the example below is from `app/(dashboard)/dashboard/useCountUp.ts`:

```
/**
 * Animates a numeric value toward a target over a fixed duration.
 * @param value The target value to count toward.
 * @param duration Animation length in milliseconds (default 900).
 * @returns The current animated value to render.
 */
export function useCountUp(value: number, duration = 900) { ... }
```

Self-explanatory helpers use a single-line JSDoc (see `app/(dashboard)/dashboard/helpers.ts` and `useCountUp.ts`). Implementation notes use `//` line comments.

Aspirational rule — file-level header JSDoc. New modules begin with a file-level JSDoc header

(File, Author, Created, Description); this is present in newer files such as `lib/constants.ts` and the extracted dashboard modules (`types.ts`, `helpers.ts`, `useCountUp.ts`). It is a goal for new files rather than a property of every existing file, and is not retrofitted across the whole codebase.

8 Formatting (Prettier)

Formatting is handled by Prettier, configured in `.prettierrc` at the repository root (Table 3).

Table 3: Prettier settings (`.prettierrc`).

Setting	Value
<code>printWidth</code>	80
<code>tabWidth</code>	2
<code>semi</code>	true
<code>singleQuote</code>	/ false (double quotes)
<code>jsxSingleQuote</code>	
<code>trailingComma</code>	es5
<code>bracketSpacing</code>	true
<code>arrowParens</code>	avoid

9 Version Control

- **Branches** are named `type/short-description`, e.g. `feature/leaderboard`, `fix/null-check-userid`, `chore/update-deps`.
- **Commit messages** use the imperative mood and describe one logical change (e.g. “Add sign-in button”), following Conventional Commits [5].
- **Quality gate.** `npm run build` and `npm run lint` must pass before a pull request is ready. (No `test` script is defined yet; automated testing is tracked in the QA Activities document.)
- **No debugging artifacts** (stray `console.log` or commented-out code) are committed.

References

- [1] Google, “Google TypeScript Style Guide,” 2024. [Online]. Available: <https://google.github.io/styleguide/tsguide.html>
- [2] Prettier, “Prettier Documentation,” 2024. [Online]. Available: <https://prettier.io/docs/en/>
- [3] ESLint, “ESLint Documentation,” 2024. [Online]. Available: <https://eslint.org/docs/latest/>
- [4] Vercel, “Next.js Documentation,” 2024. [Online]. Available: <https://nextjs.org/docs>
- [5] Conventional Commits, “Conventional Commits v1.0.0,” 2023. [Online]. Available: <https://www.conventionalcommits.org/>